



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems Lecture 23

File System 4: reliability & distributed systems

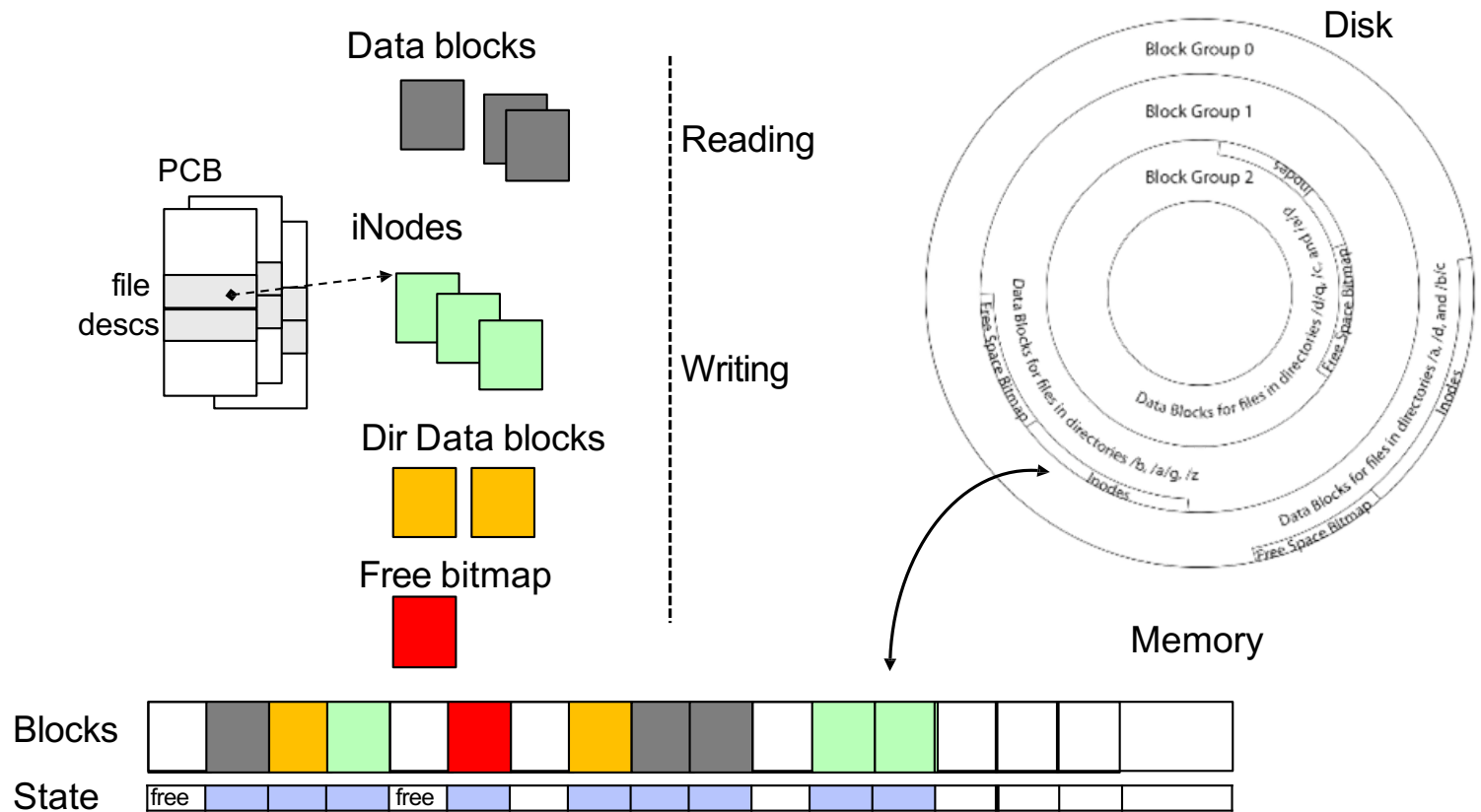
**Prof. Yunming XIAO
School of Data Science**

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

Recall

Recall: file system buffer cache

OS implements a cache of disk blocks for efficient access to data, directories, inodes, freemap



Recall: storage reliability problem

- Blocks written to buffer cache are flushed to disk periodically, but may be lost if your machine crashes before that
- Even without a buffer cache, file system data structures may be inconsistent after a failure
 - Single logical file operation can involve updates to multiple disk blocks (inode, indirect block, data block, bitmap, ...)
 - Within the storage device, single block write might update multiple sectors
- How do we guarantee consistency regardless of when a crash occurs?

Recall: two reliability approaches

Careful Ordering and Recovery

FAT & FFS + (fsck)

Each step builds structure,

Data block $\Leftarrow$ inode $\Leftarrow$ free $\Leftarrow$ directory

Last step links it in to rest of FS

“Recover” operation scans structure
looking for incomplete actions

Versioning and Copy-on-Write

ZFS, ...

Version files at some granularity

Create new structure linking back to
unchanged parts of old one

Last step is to declare that the new
version is ready

Reliability approach #1: careful ordering

- Sequence operations in a specific order
 - Careful design to allow sequence to be interrupted safely
- Post-crash recover operation
 - Read data structures to see if there were any operations in progress
 - Clean up/finish as needed
- Approach taken by
 - FAT and FFS (fsck) to protect filesystem structure/metadata
 - Many app-level recovery schemes (e.g., Word, emacs autosaves)

Berkeley FFS: create a file

Normal operation:

- Allocate data block
- Write data block
- Allocate inode
- Write inode block
- Update bitmap of free blocks and inodes
- Update directory with file name → inode number
- Update modify time for directory

Recovery:

- Scan inode table
- If any unlinked files (not in any directory), delete or put in lost & found dir
- Compare free block bitmap against inode trees
- Scan directories for missing update/access times

Time proportional to disk size

Reliability approach #2: copy-on-write file layout

- Create a new version of the file with the updated data
 - Reuse blocks that don't change much of what is already in place
 - Only point to the new version when fully done writing it
- Seems expensive!
 - But updates can be batched, and many disk writes can occur in parallel
- Approach taken in network file server appliances
 - NetApp's Write Anywhere File Layout (WAFL)
 - ZFS (Sun/Oracle) and OpenZFS

More general reliability solutions

- Use *transactions* for atomic updates
 - Ensure that multiple related updates are performed atomically
 - I.e., if a crash occurs in the middle, the state of the systems reflects either all or none of the updates
 - Most modern file systems use transactions internally to update metadata
 - Many applications also implement their own transactions
- Use *redundancy* for media failures
 - Redundant representation on one storage device (Error Correcting Codes)
 - Replication across devices (e.g., RAID disk array)

Transactions

- Closely related to critical sections for manipulating shared data structures
- They extend concept of atomic update from memory to stable storage
 - Atomically update multiple persistent data structures
- Many ad-hoc approaches
 - FFS carefully ordered the sequence of updates so that if a crash occurred while manipulating directory or inodes, the disk scan on reboot would detect and recover the error (fsck)
 - Applications use temporary files and rename

Key concept: transaction

- A *transaction* is an atomic sequence of reads and writes that takes the system from one consistent state to another.



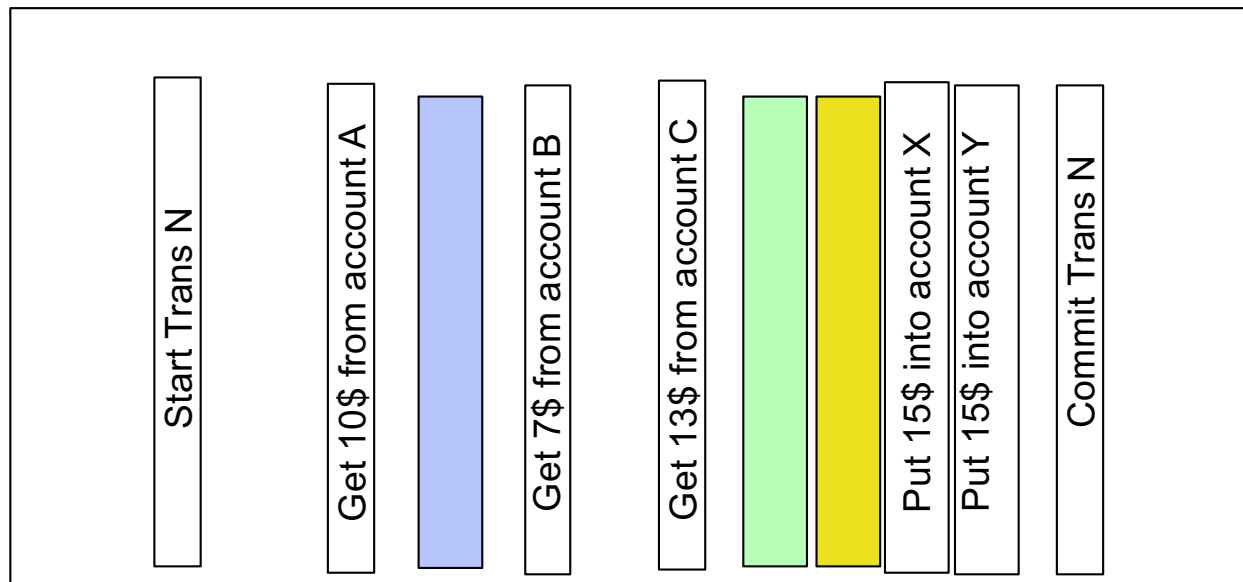
- Recall: Code in a critical section appears atomic to other threads
- Transactions extend the concept of atomic updates from *memory* to *persistent storage*

Typical structure

- **Begin** a transaction – get a transaction ID
- Do a bunch of updates
 - If any fail along the way, **roll-back**
 - Or, if any conflicts with other transactions, **roll-back**
- **Commit** the transaction

Useful tool: a log

- One simple action is atomic – write/append a basic item
- Use that to seal the commitment to a whole series of actions



Transactional file systems

- Better reliability through use of log
 - Changes to all FS data structures are treated as transactions
 - A transaction is committed once it is fully written to the log
 - Data forced to disk for reliability
 - Process can be accelerated with NVRAM
 - Although the actual file system data structures may not be updated immediately, data preserved in the log and replayed to recover
- Most common: *journaling file systems*

Journaling file systems

- Don't modify data structures on disk directly
- Write each update as transaction recorded in a log
 - Commonly called a journal or intention list
 - Also maintained on disk (allocate specific blocks for it)
- Once changes are in the log, they can be safely applied to other data structures
 - E.g. modify inode pointers and directory entries
- Linux took original FFS-like file system (ext2) and added a journal to get ext3!
 - Some options: whether or not to write all data to journal or just metadata

Journaling summary

Why go through all this trouble?

- Updates atomic, even if we crash:
 - Update either gets fully applied or discarded
 - All physical operations *treated as a logical unit*

Isn't this expensive?

- Yes! We're now writing all data twice (once to log, once to actual data blocks in target file)
- Modern filesystems journal metadata updates only
 - Record modifications to file system data structures
 - But apply updates to a file's contents directly

Distributed systems

What is a distributed system?

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

-- Leslie Lamport

Centralized vs distributed systems

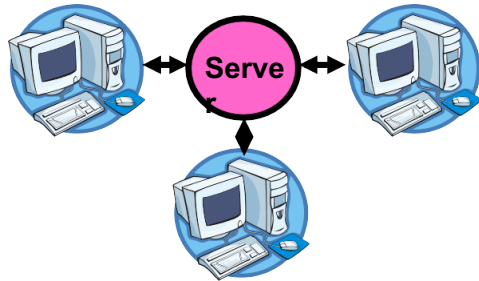


So far, considered a single computer! All computation was done on the local computer in isolation



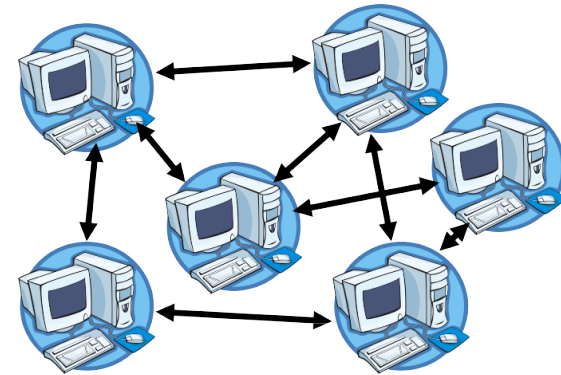
The world is a large distributed system

Two types of distributed systems



Client/Server Model

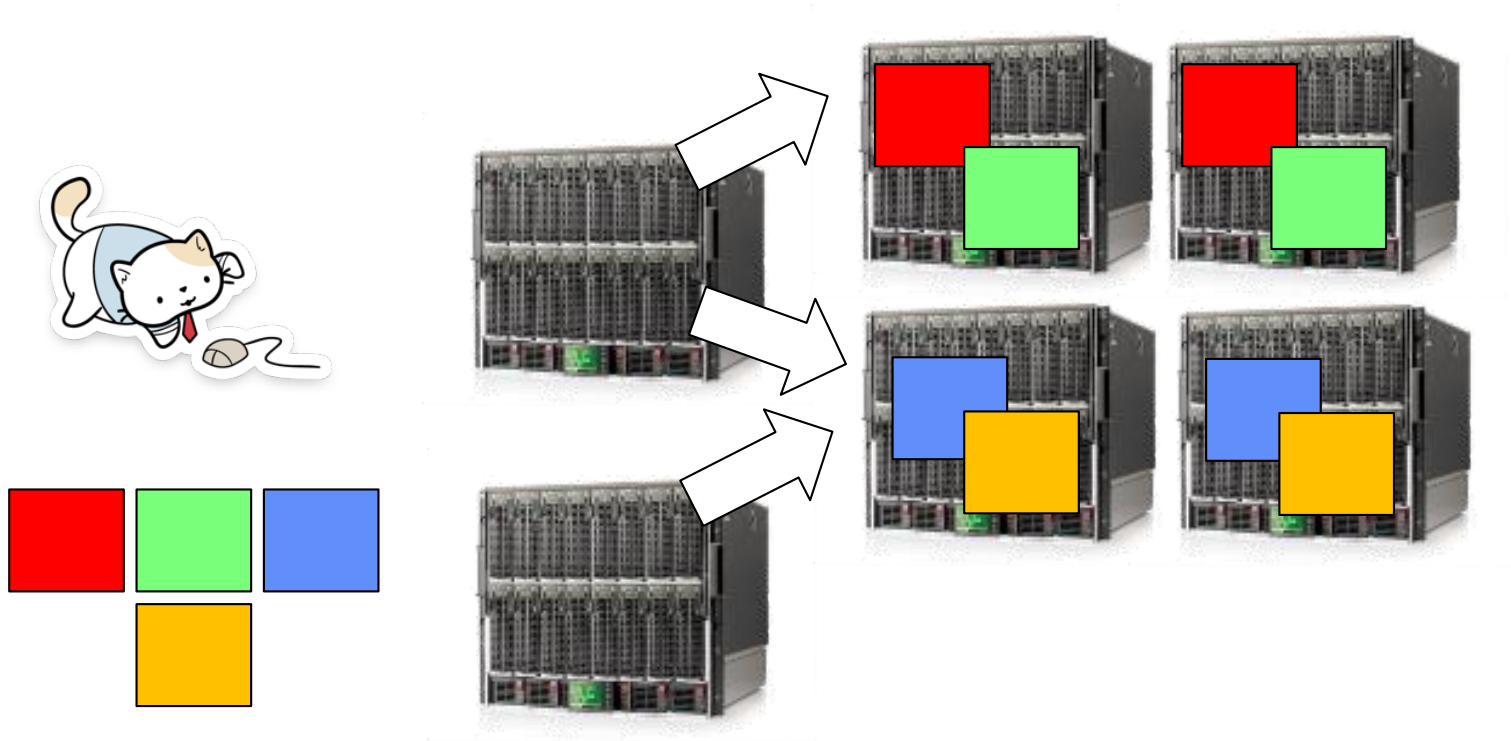
- One or more server provides *services* to clients
- Clients make *remote procedure calls* to server
- Server serves *requests* from clients



Peer-to-Peer Model

- Each computer acts as a peer
- No hierarchy or central point of coordination
- All-way communication between peers through *gossiping*

Example: How do I store all my data?



The promise of distributed systems

- Availability
 - Proportion of time system is in functioning condition
⇒ One machine goes down, use another
- Fault-tolerance
 - System has well-defined behavior when fault occurs
⇒ Store data in multiple locations
- Scalability
 - Ability to add resources to system to support more work
⇒ Just add machines when need more storage/processing power
- Transparency
 - The ability of the system to mask its complexity behind a simple interface

Transparency

- **Location:** Can't tell where resources are located
- **Migration:** Resources may move without the user knowing
- **Replication:** Can't tell how many copies of resource exist
- **Concurrency:** Can't tell how many users there are
- **Parallelism:** System may speed up large jobs by splitting them into smaller pieces
- **Fault Tolerance:** System may hide various things that go wrong

The challenges of distributed systems

- How do you get machines to **communicate**?
- How do you get machines to **coordinate**?
- How do you deal with **failures**?
- How do you deal with **security** (corrupted machines)?

Topic roadmap

Distributed File System: NFS

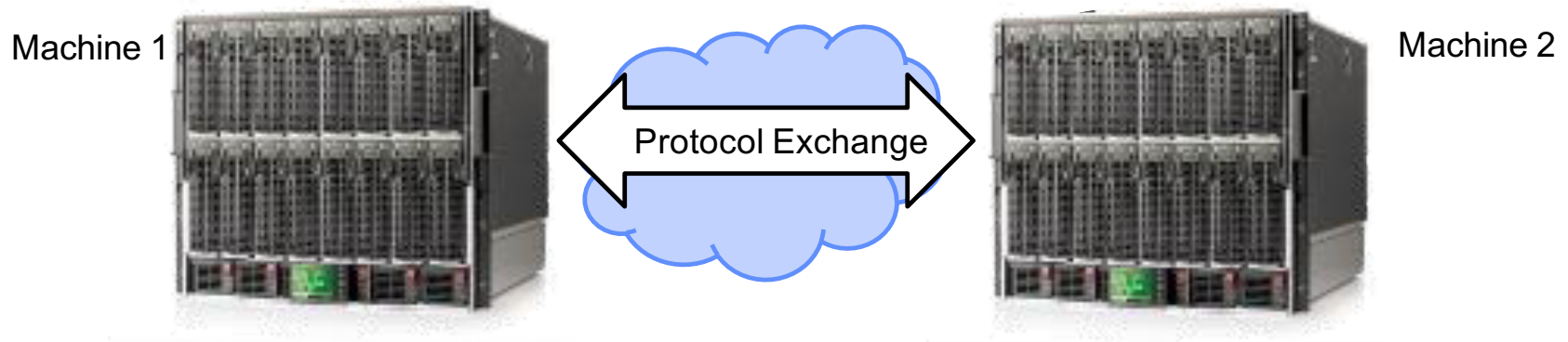
Peer-To-Peer System: The Internet

Distributed Data Processing
(MapReduce and Spark)

Coordination
(Atomic Commit and Consensus)

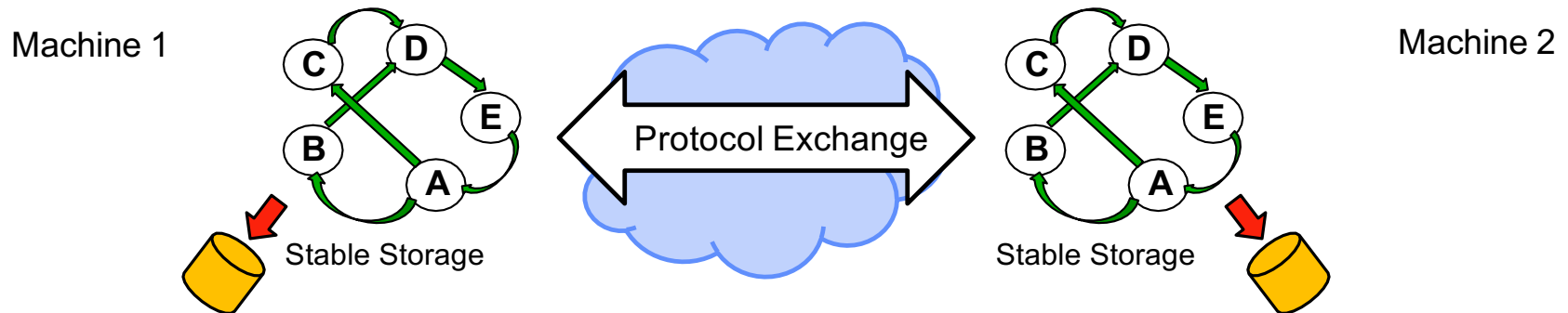
Message passing

How do machines communicate?



- A protocol is **an agreement on how to communicate**, covering
 - **Syntax**: how a communication is specified & structured
 - Format, order messages are sent and received
 - **Semantics**: what a communication means
 - Actions taken when transmitting, receiving, or when a timer expires

How do machines communicate?



- A protocol is **an agreement on how to communicate**, covering
 - **Syntax**: how a communication is specified & structured
 - Format, order messages are sent and received
 - **Semantics**: what a communication means
 - Actions taken when transmitting, receiving, or when a timer expires

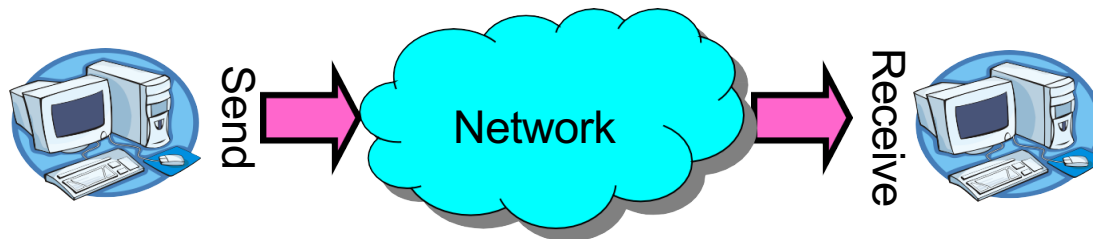
Examples of protocols in human interactions

Telephone

1. (Pick up / open up the phone)
2. Listen for a dial tone / see that you have service
3. Dial
4. Should hear ringing ...
5.  Callee: "Hello?"
6. Caller: "Hi, it's Matei...."
Or: "Hi, it's me" (← what's *that* about?) 
7. Caller: "Hey, do you think ... blah blah blah ..." **pause**
8.  Callee: "Yeah, blah blah blah ..." **pause**
9. Caller: Bye 
10.  Callee: Bye
11. Hang up 

Message passing

- How do you actually program a distributed application?



- Interface:
 - Mailbox (mbox): temporary holding area for messages
 - `Send(message,mbox)`
 - `Receive(buffer,mbox)`

Question: data representation

- An object in memory has a **machine-specific binary representation**
- Without shared memory, externalizing an object requires us to turn it into a sequential sequence of bytes
 - **Serialization/Marshalling**: Express an object as a sequence of bytes
 - **Deserialization/Unmarshalling**: Reconstructing the original object from its marshalled form at destination

Simple data types

```
uint32_t x;
```

- Suppose I want to write a x to a file

- First, open the file:

```
FILE* f = fopen("foo.txt", "w");
```

- Then, I have two choices:

```
1. fprintf(f, "%lu", x);
```

```
2. fwrite(&x, sizeof(uint32_t), 1, f);
```

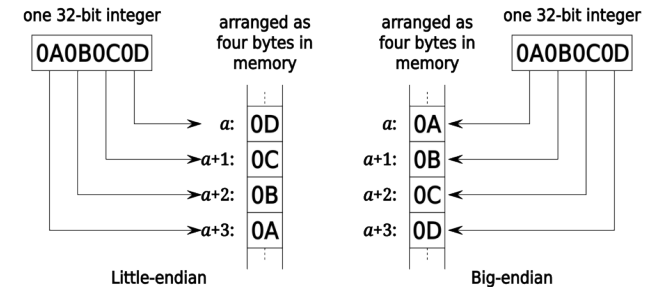
- Neither one is “wrong” but sender and receiver should be consistent!

Machine representation: endianness

What order are the bytes of a multi-byte integer stored in physical memory?

Big Endian: most significant byte is stored first

Little Endian: least significant byte is stored first



```
int val = 0x12345678;
printf("val = %x\n", val);

int i = 0;
for (i = 0; i < sizeof(val); i++) {
    printf("val[%d] = %x\n", i, ((uint8_t *) &val)[i]);
}
```

→ code `./lec23-endian`

```
val = 12345678
val[0] = 78
val[1] = 56
val[2] = 34
val[3] = 12
```

Processor	Endianness
Motorola 68000	Big Endian
PowerPC (PPC)	Big Endian
Sun Sparc	Big Endian
IBM S/390	Big Endian
Intel x86 (32 bit)	Little Endian
Intel x86_64 (64 bit)	Little Endian
Dec VAX	Little Endian
Alpha	Bi (Big/Little) Endian
ARM	Bi (Big/Little) Endian
IA-64 (64 bit)	Bi (Big/Little) Endian
MIPS	Bi (Big/Little) Endian

What about more complex objects?

- Consider word_count_t of Homework 0 and 1 ...
- Each element contains:
 - An int
 - A *pointer* to a string (of some length)
 - A *pointer* to the next element
- fprintf_words writes these as a sequence of lines (character strings with \n) to a file stream
- What if you wanted to write the whole list as a binary object (and read it back as one)?
 - How do you represent the string?
 - Does it make any sense to write the pointer?

```
typedef struct word_count
{
    char *word;
    int count;
    struct word_count *next;
}
word_count_t;
```

Data serialization formats

- JSON, XML and Protocol Buffers are commonly used in web applications
- Lots of ad-hoc formats

```
{ "faculty":  
  [  
    { "id": 1,  
      "firstname": "Ion",  
      "lastname": "Stoica"  
    },  
    { "id": 2,  
      "firstname": "Matei",  
      "lastname": "Zaharia"  
    }  
  ]  
}
```

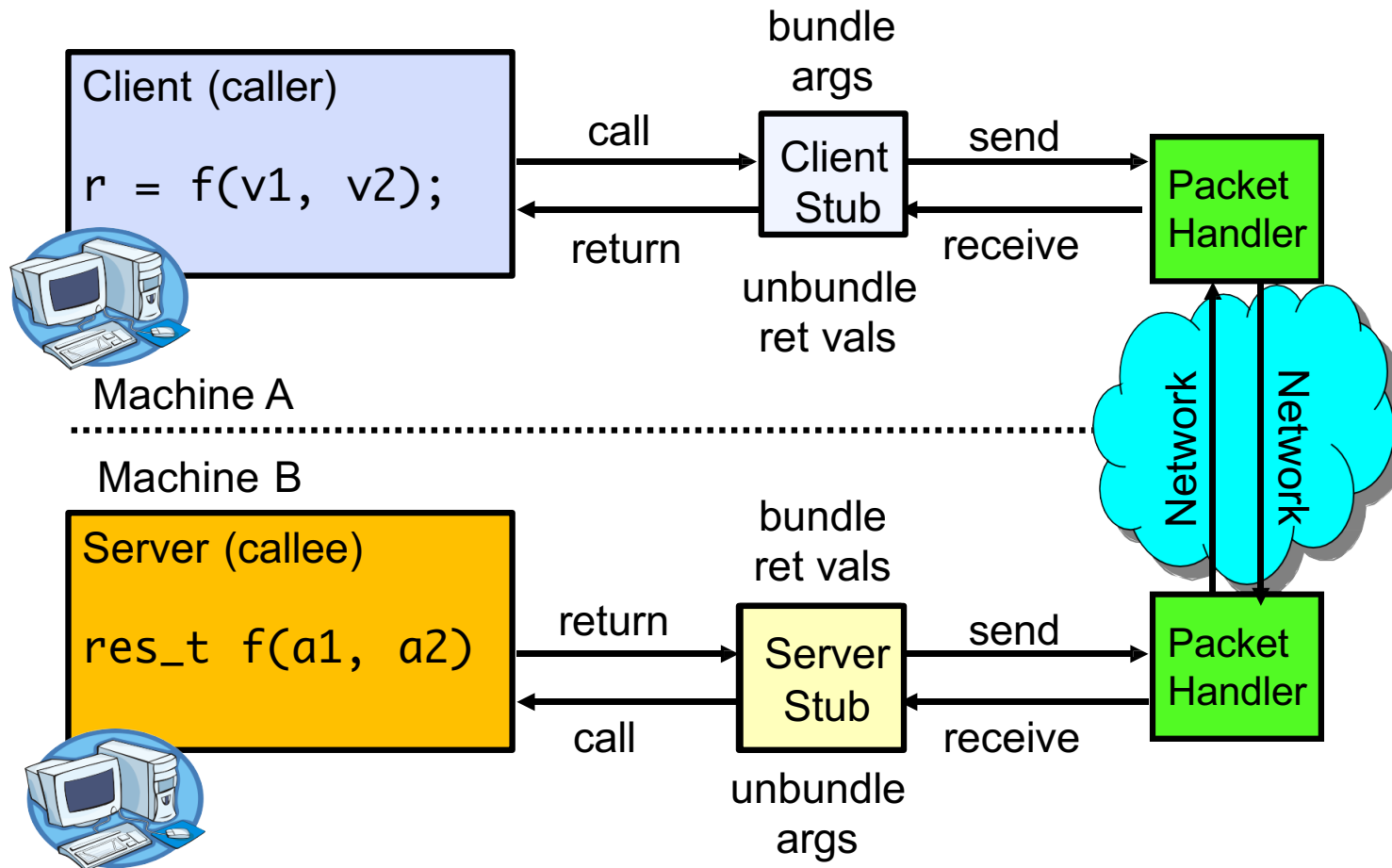
Data serialization formats

Name	Creator-maintainer	Based on	Standardized?	Specification	Binary?	Human-readable?	Supports references?	Schema-IDL?	Standard APIs	Supports Zero-copy operations
Apache Avro	Apache Software Foundation	N/A	No	Apache Avro™ 1.8.1 Specification	Yes	No	N/A	Yes (built-in)	N/A	N/A
Apache Parquet	Apache Software Foundation	N/A	No	Apache Parquet	Yes	No	No	N/A	Java, Python	No
ASN.1	ISO, IEC, ITU-T	N/A	Yes	ISO/IEC 8824: X.680 series of ITU-T Recommendations	Yes (BER, DER, PER, OER, or custom via ECN)	Yes (XER, JER, GSER, or custom via ECN)	Partial	Yes (built-in)	N/A	Yes (OER)
Bencode	Bram Cohen (creator) BitTorrent, Inc. (maintainer)	N/A	De facto standard via BitTorrent Enhancement Proposal (BEP)	Part of BitTorrent protocol specification	Partially (numbers and delimiters are ASCII)	No	No	No	No	N/A
Binn	Bernardo Ramos	N/A	No	Binn Specifications	Yes	No	No	No	No	Yes
BSON	MongoDB	JSON	No	BSON Specification	Yes	No	No	No	No	N/A
CBOR	Carsten Bormann, P. Hoffman	JSON (loosely)	Yes	RFC 7049	Yes	No	Yes through tagging	Yes (CODL)	No	Yes
Comma-separated values (CSV)	RFC author: Yakov Shafranovich	N/A	Partial (myriad informal variants used)	RFC 4180 (among others)	No	Yes	No	No	No	No
Common Data Representation (CDR)	Object Management Group	N/A	Yes	General Inter-ORB Protocol	Yes	No	Yes	Yes	ADA, C, C++, Java, Cobol, Lisp, Python, Ruby, Smalltalk	N/A
D-Bus Message Protocol	freedesktop.org	N/A	Yes	D-Bus Specification	Yes	No	No	Partial (Signature strings)	Yes (see D-Bus)	N/A
Efficient XML Interchange (EXI)	W3C	XML, Efficient XML	Yes	Efficient XML Interchange (EXI) Format 1.0	Yes	Yes (XML)	Yes (XPath)	Yes (XML Schema)	Yes (DOM, SAX, XPath)	N/A
FlatBuffers	Google	N/A	No	flatbuffers github page	Yes	Yes (Apache Arrow)	Partial (internal to the buffer)	Yes (2)	C++, Java, C#, Go, Python, Rust, JavaScript, PHP, C, Dart, Lua, TypeScript	Yes
Fast Infoset	ISO, IEC, ITU-T	XML	Yes	ITU-T X.891 and ISO/IEC 24824-1:2007	Yes	No	Yes (XPath)	Yes (XML schema)	Yes (DOM, SAX, XPath)	N/A
FHIR	Health_Level_7	REST basics	Yes	Fast Healthcare Interoperability Resources	Yes	Yes	Yes	Yes	Hapi for FHIR ^[1] JSON, XML, Turtle	No
Ion	Amazon	JSON	No	The Amazon Ion Specification	Yes	Yes	No	No	No	N/A
Java serialization	Oracle Corporation	N/A	Yes	Java Object Serialization	Yes	No	Yes	No	Yes	N/A
JSON	Douglas Crockford	JavaScript syntax	Yes	STD 90/RFC 8259 (ancillary: RFC 6901, RFC 6902), ECMA-404, ISO/IEC 21778:2017	No, but see BSON, Smile, UBJSON	Yes	Yes (JSON Pointer (RFC 6901); alternately: JSONPath, XPath, JSPON, json.selected), JSON-LD	Partial (JSON Schema Proposal, ASN.1 with JER, Kwalify, RxJS, Javascript Schemas), JSON-LD	Partial (Clarinet, JSONQuery, JSONPath), JSON-LD	No
MessagePack	Sadayuki Furuhashi	JSON (loosely)	No	MessagePack format specification	Yes	No	No	No	No	Yes
Netstrings	Dan Bernstein	N/A	No	netstrings.txt	Yes	Yes	No	No	No	Yes
OGDL	Rolf Veen	?	No	Specification	Yes (Binary Specification)	Yes	Yes (Path Specification)	Yes (Schema WD)		N/A
OPC-UA Binary	OPC Foundation	N/A	No	opcfoundation.org	Yes	No	Yes	No	No	N/A
OpenDDL	Eric Lengyel	C, PHP	No	OpenDDL.org	No	Yes	Yes	No	Yes (OpenDDL Library)	N/A
Pickle (Python)	Guido van Rossum	Python	De facto standard via Python Enhancement Proposals (PEPs)	[3] PEP 3154 -- Pickle protocol version 4	Yes	No	No	No	Yes ([4])	No
Property list	NeXT (creator) Apple (maintainer)	?	Partial	Public DTD for XML format	Yes ^a	Yes ^b	No	?	Cocoa, CoreFoundation, OpenStep, GNUStep	No
Protocol Buffers (protobuf)	Google	N/A	No	Developer Guide: Encoding	Yes	Partial ^d	No	Yes (built-in)	C++, C#, Java, Python, Javascript, Go	No
	John McFarlane					Yes				

Remote Procedure Call (RPC)

- Raw messaging is a bit too low-level for programming
 - Must wrap up information into message at source
 - Must decide what to do with message at destination
 - May need to sit and wait for multiple messages to arrive
 - And must deal with machine representation by hand
- Common abstraction: **Remote Procedure Call (RPC)**
 - Calls a procedure on a remote machine
 - Idea: Make communication look like an ordinary function call
 - Automate all of the complexity of translating between representations
 - Client calls:
 - `remoteFileSystem.read("rutabaga.txt");`
 - Translated automatically into call on server:
 - `fileSys.read("rutabaga.txt");`

RPC information flow



RPC implementation

- Request-response message passing (under covers!)
- “Stub” provides glue on client/server
 - Client stub is responsible for “marshalling” arguments and “unmarshalling” the return values
 - Server-side stub is responsible for “unmarshalling” arguments and “marshalling” the return values.
- Marshalling involves (depending on system)
 - Converting values to a canonical form, serializing objects, copying arguments passed by reference, etc.

RPC details (1/3)

- Equivalence with regular procedure call
 - Parameters $\Leftrightarrow$ Request Message
 - Result $\Leftrightarrow$ Reply message
 - Name of Procedure: Passed in request message
 - Return Address: mbox2 (client return mail box)
- Stub generator: Compiler that generates stubs
 - Input: interface definitions in an “interface definition language (IDL)”
 - Contains, among other things, types of arguments/return
 - Output: stub code in the appropriate source language
 - Code for client to pack message, send it, wait for result, unpack result and return to caller
 - Code for server to unpack message, call procedure, pack results, send them

RPC details (2/3)

- Cross-platform issues:
 - What if client/server machines are different architectures/ languages?
 - Convert everything to/from some canonical form
 - Tag every item with an indication of how it is encoded (avoids unnecessary conversions)
- How does client know which mbox (destination queue) to send to?
 - Need to translate name of remote service into network endpoint (Remote machine, port, possibly other info)
 - *Binding*: the process of converting a user-visible name into a network endpoint
 - This is another word for “naming” at network level
 - Static: fixed at compile time
 - Dynamic: performed at runtime

RPC details (3/3)

- Dynamic Binding
 - Most RPC systems use dynamic binding via name service
 - Name service provides dynamic translation of service → mbox
 - Why dynamic binding?
 - Access control: check who is permitted to access service
 - Fail-over: If server fails, use a different one
- What if there are multiple servers?
 - Could give flexibility at binding time
 - Choose unloaded server for each new client
 - Could provide same mbox (router level redirect)
 - Choose unloaded server for each new request
 - Only works if no state carried from one call to next
- What if multiple clients?
 - Pass pointer to client-specific return mbox in request

Problems with RPC: non-atomic failures

- Different failure modes in dist. system than on a single machine
- Consider many different types of failures
 - User-level bug causes address space to crash
 - Machine failure, kernel bug causes all processes on same machine to fail
 - Some machine is compromised by malicious party
- Can easily result in inconsistent view of the world
 - Did my cached data get written back or not?
 - Did server do what I requested or not?
- Answer? **Distributed transactions / 2PC**

Problems with RPC: performance

- RPC is *not* performance transparent:
 - Cost of Procedure call << same-machine RPC << network RPC
 - Overheads: Marshalling, Stubs, Kernel-Crossing, Communication
- Programmers must be aware that RPC is not free
 - Caching can help, but may make failure handling even harder

Distributed file systems

Topic roadmap

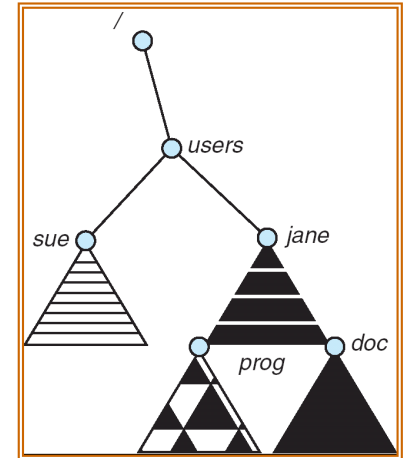
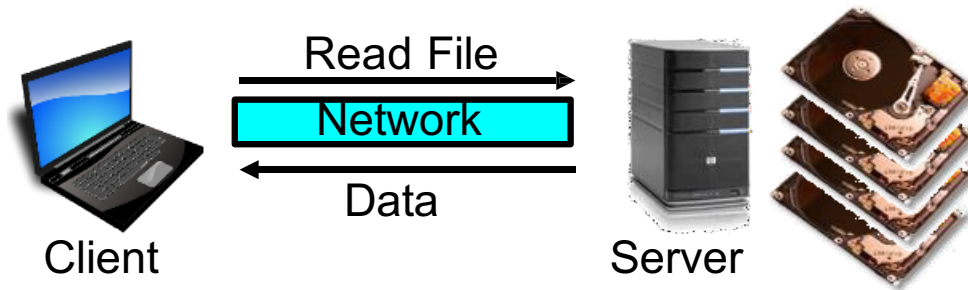
Distributed File System: NFS

Peer-To-Peer System: The Internet

Distributed Data Processing
(MapReduce and Spark)

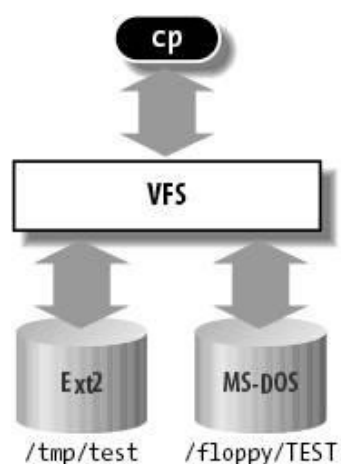
Coordination
(Atomic Commit and Consensus)

Distributed File Systems



- Transparent access to files stored on a remote disk
- *Mount* remote files into your local file system
 - Directory in local file system refers to remote files
 - e.g., `/users/jane/prog/foo.c` on laptop actually refers to `/prog/foo.c` on `fs.cs.berkeley.edu`
- Naming Choices:
 - [Hostname,localname]: Filename includes server
 - A global name space: Filename unique in “world”

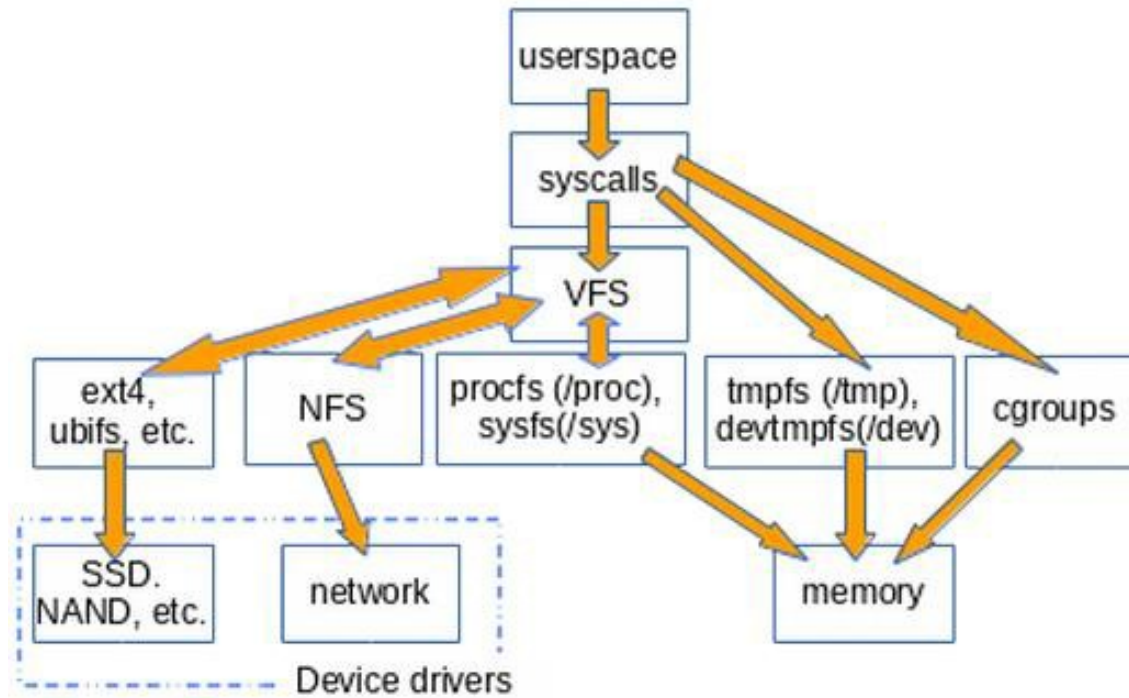
Virtual Filesystem Switch (VFS)



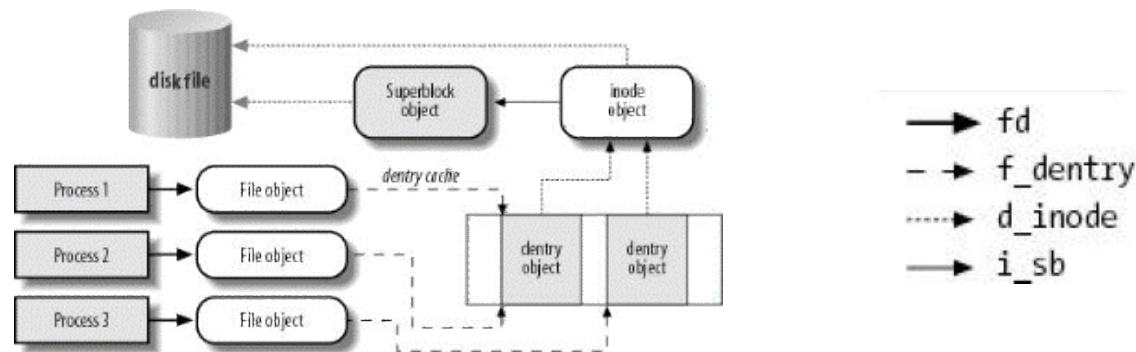
```
inf = open("/floppy/TEST", O_RDONLY, 0);
outf = open("/tmp/test",
            O_WRONLY|O_CREAT|O_TRUNC, 0600);
do {
    i = read(inf, buf, 4096);
    write(outf, buf, i);
} while (i);
close(outf);
close(inf);
```

- Virtual abstraction of file system in many OSes (including Linux)
 - Provides virtual superblocks, inodes, files, etc
 - Compatible with a variety of local and remote file systems
- VFS allows the same system call interface (the API) to be used for different types of file systems in the same naming hierarchy
 - The API is to the VFS layer, rather than any specific type of file system

Example Linux mounting tree

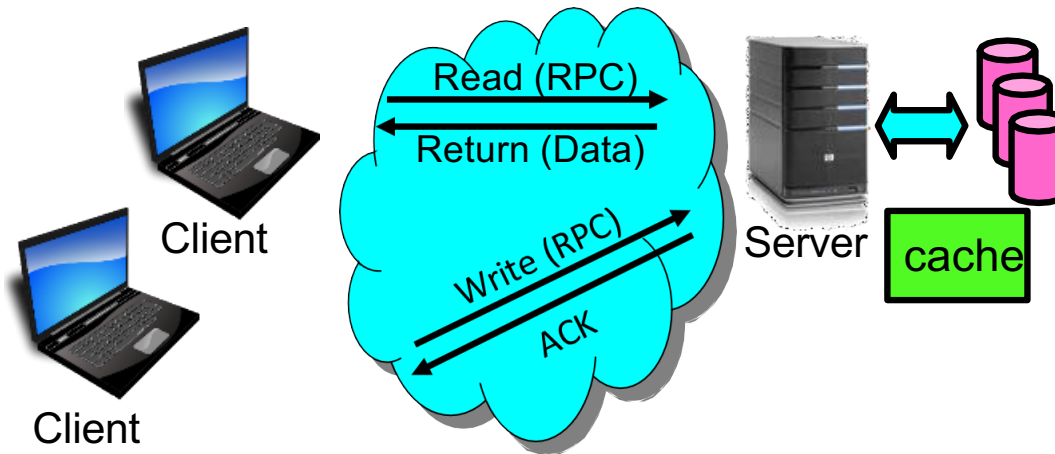


VFS common file model in Linux



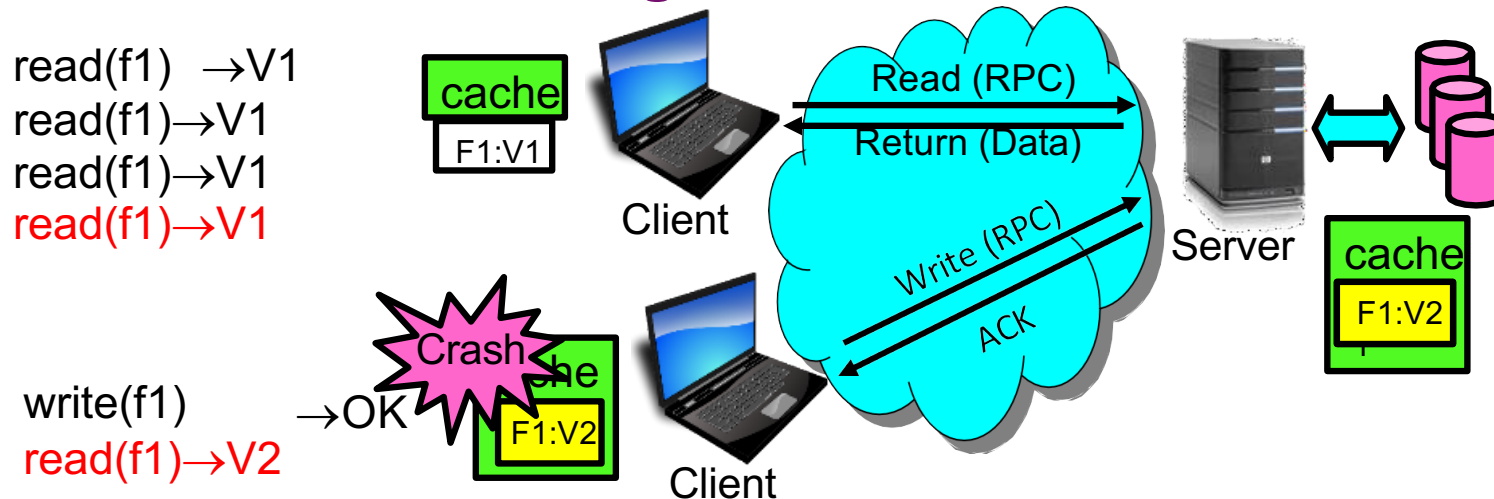
- Four primary object types for VFS:
 - superblock object: represents a specific mounted filesystem
 - inode object: represents a specific file
 - dentry object: represents a directory entry
 - file object: represents open file associated with process
- May need to fit the model by faking it

Simple distributed file system



- Remote Disk: Reads and writes forwarded to server. Use Remote Procedure Calls (RPC) to translate file system calls into remote requests
- Advantage: Server provides consistent view of file system to multiple clients
- Problems? Performance!
 - Going over network is slower than going to local memory
 - Lots of network traffic/not well pipelined
 - Server can be a bottleneck

Use of caching to reduce network load



- Idea: Use caching to reduce network load
 - In practice: use buffer cache at source and destination
- Advantage: if open/read/write/close can be done locally, don't need to do any network traffic...fast!
- Problems:
 - Failure: Client caches have data not committed at server
 - Cache consistency! Client caches not consistent with server/each other

Dealing with failures

- What if the server crashes? Can client wait until it comes back and just continue making requests?
 - Changes in server's buffer cache but not in disk are lost
- What if there is shared state across RPCs?
 - Client opens file, then does a seek
 - Server crashes
 - What if client wants to do another read?
- Similar problem: What if client removes a file but server crashes before acknowledgement?

Stateless protocols

- **Stateless Protocol:** A protocol in which all information required to service a request is included with the request
- **Idempotent Operations** – repeating an operation multiple times is same as executing it just once (e.g., storing to a mem addr.)
- Client: timeout expires without reply, just run the operation again (safe regardless of first attempt)
- HTTP is also a stateless protocol!
 - Include cookies with request to simulate a session

Case study: Network File System (NFS)

- Introduced by Sun in 1986 as an open protocol
- Three layers for NFS system
 - **UNIX file-system interface**: open, read, write, close calls + file descriptors
 - **VFS layer**: distinguishes local from remote files
 - Calls the NFS protocol procedures for remote requests
 - **NFS service layer**: bottom layer of the architecture
 - Implements the NFS protocol

NFS continued

- NFS protocol: RPC for file operations on server
 - Reading/searching a directory
 - Manipulating links and directories
 - Accessing file attributes/reading and writing files
- Write-through caching: Modified data committed to server's disk before returning a result to the client
 - Lose some of the advantages of caching
 - Time to perform write() can be long
 - Need some way for readers to eventually notice changes! (more on this later)

NSFv2: every write persisted

NSFv3: open-to-close consistency

NFS continued

- NFS servers are **stateless**; each request provides all arguments require for execution
 - E.g. reads include information for entire operation, such as `ReadAt(inumber,position)`, not `Read(openfile)`
 - No need to perform network `open()` or `close()` on file; each operation stands on its own
- **Idempotent**: Performing requests multiple times has same effect as performing them exactly once
 - Example: Read and write file blocks: just re-read or re-write file block – no other side effects
 - Example: What about “remove”? NFS does operation twice and second time returns an advisory error

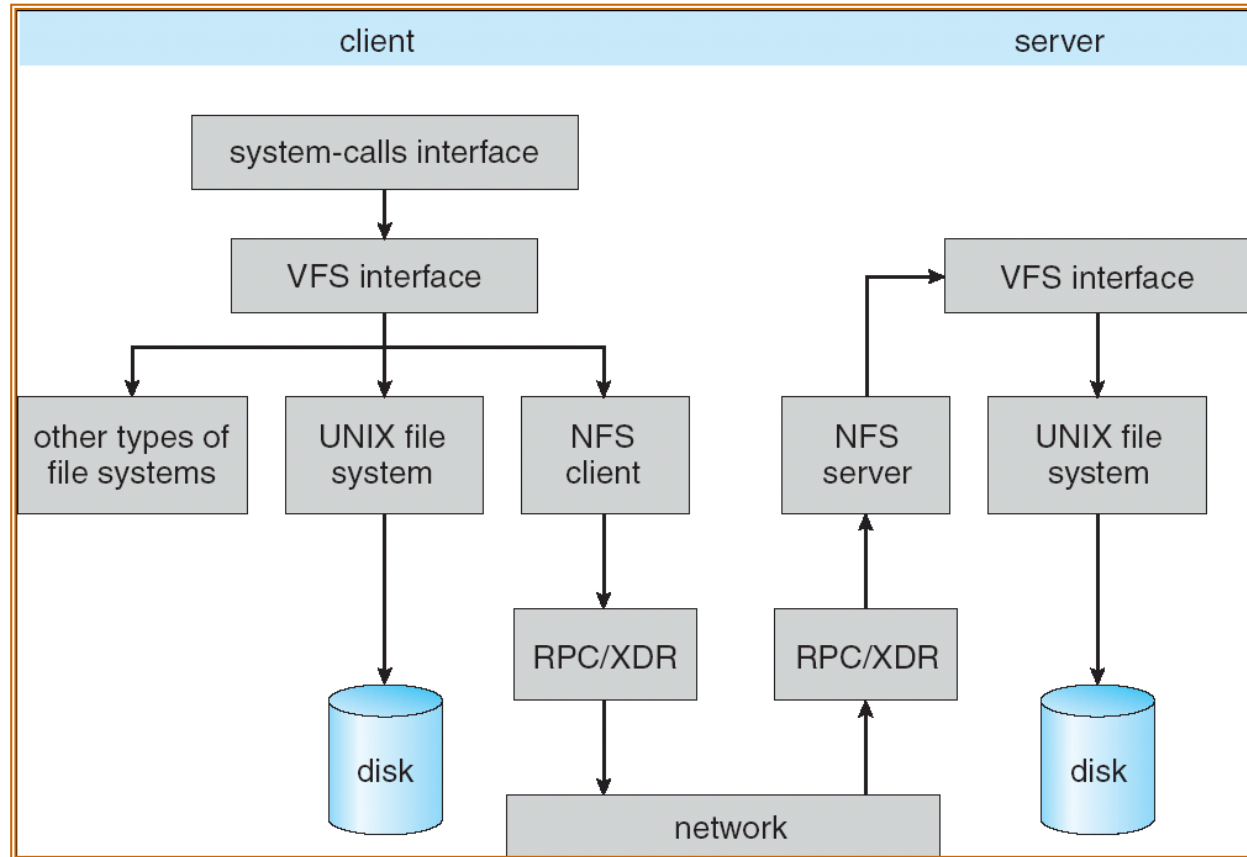
NFS continued

NFSPROC_GETATTR	file handle returns: attributes
NFSPROC_SETATTR	file handle, attributes returns: –
NFSPROC_LOOKUP	directory file handle, name of file/dir to look up returns: file handle
NFSPROC_READ	file handle, offset, count data, attributes
NFSPROC_WRITE	file handle, offset, count, data attributes
NFSPROC_CREATE	directory file handle, name of file, attributes –
NFSPROC_REMOVE	directory file handle, name of file to be removed –
NFSPROC_MKDIR	directory file handle, name of directory, attributes file handle
NFSPROC_RMDIR	directory file handle, name of directory to be removed –
NFSPROC_READDIR	directory handle, count of bytes to read, cookie returns: directory entries, cookie (to get more entries)

NFS continued

Client	Server
fd = open("/foo", ...); Send LOOKUP (rootdir FH, "foo")	Receive LOOKUP request look for "foo" in root dir return foo's FH + attributes
Receive LOOKUP reply allocate file desc in open file table store foo's FH in table store current file position (0) return file descriptor to application	
read(fd, buffer, MAX); Index into open file table with fd get NFS file handle (FH) use current file position as offset Send READ (FH, offset=0, count=MAX)	Receive READ request use FH to get volume/inode num read inode from disk (or cache) compute block location (using offset) read data from disk (or cache) return data to client
Receive READ reply update file position (+bytes read) set current file position = MAX return data/error code to app	
read(fd, buffer, MAX); Same except offset=MAX and set current file position = 2*MAX	

NFS architecture



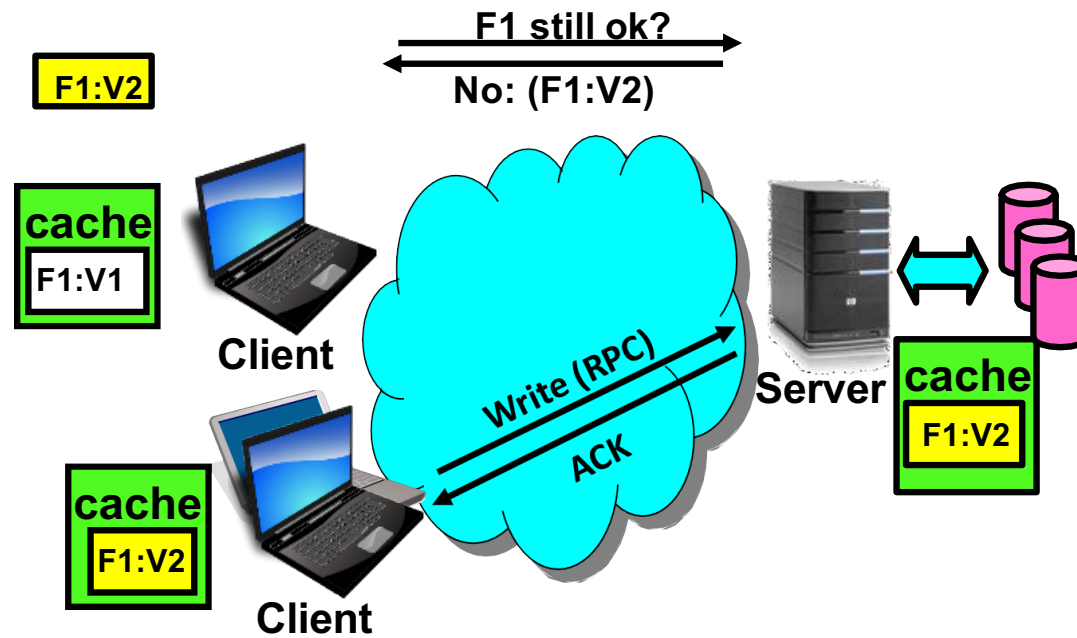
NFS failure model

- Failure Model: Transparent to client system
 - Is this a good idea? What if you are in the middle of reading a file and server crashes?
 - Options (NFS Provides both):
 - Hang until server comes back up (next week?)
 - Return an error.
(Of course, most applications don't know they are talking over network)

NFS cache consistency

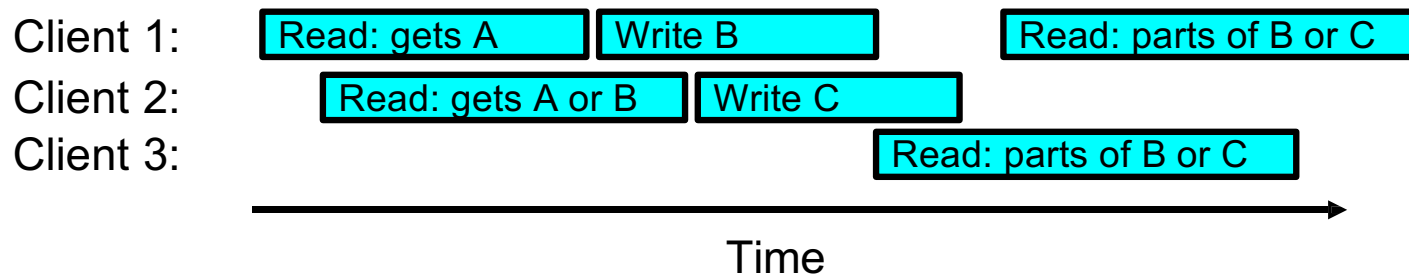
- NFS protocol: weak consistency
 - Client polls server periodically to check for changes
 - Polls server if data hasn't been checked in last 3-30 seconds (exact timeout is tunable parameter).
 - Thus, when file is changed on one client, server is notified, but other clients use old version of file until timeout.
- What if multiple clients write to same file?
 - In NFS, can get either version (or parts of both)
 - Completely arbitrary!

NFS cache consistency



Sequential ordering constraints

- What sort of cache coherence might we expect?
 - i.e. what if one CPU changes file, and before it's done, another CPU reads file?
- Example: Start with file contents = "A"



Sequential ordering constraints

- What would we actually want?
 - Assume we want distributed system to behave exactly the same as if all processes are running on single system
 - If read finishes before write starts, get old copy
 - If read starts after write finishes, get new copy
 - Otherwise, get either new or old copy
- For NFS:
 - If read starts more than 30 seconds after write, get new copy; otherwise, could get partial update

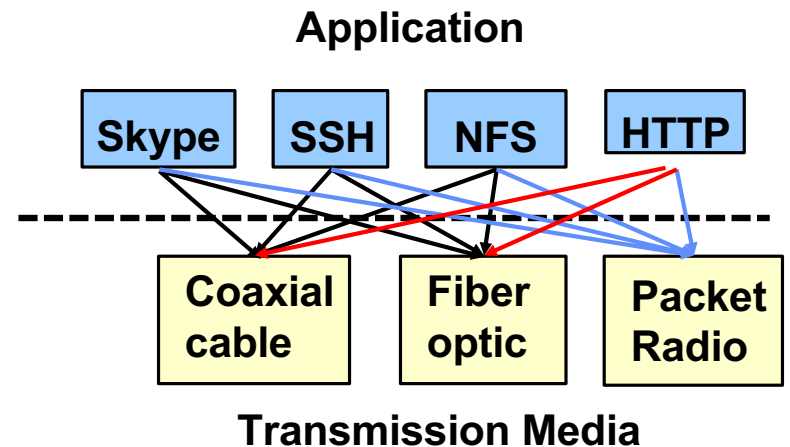
NFS pros and cons

- NFS pros:
 - Simple, Highly portable
- NFS cons:
 - Sometimes inconsistent!
 - Doesn't scale to large # clients
 - Must keep checking to see if caches out of date
 - Server becomes bottleneck due to polling traffic

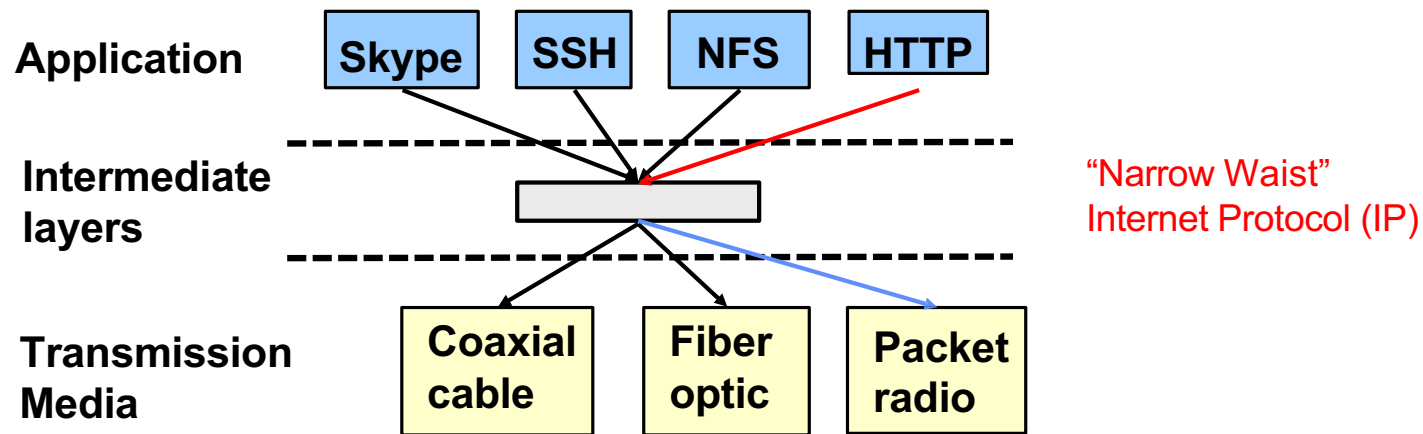
The Internet

Case study: the Internet

- The Internet is the largest distributed system that exists!
- Many different applications
 - Email, web, P2P, etc.
- Many different operating systems and devices
- Many different network styles and technologies
 - Wireless, wired, optical
- How do we organize this mess
 - Layering & end-to-end principle

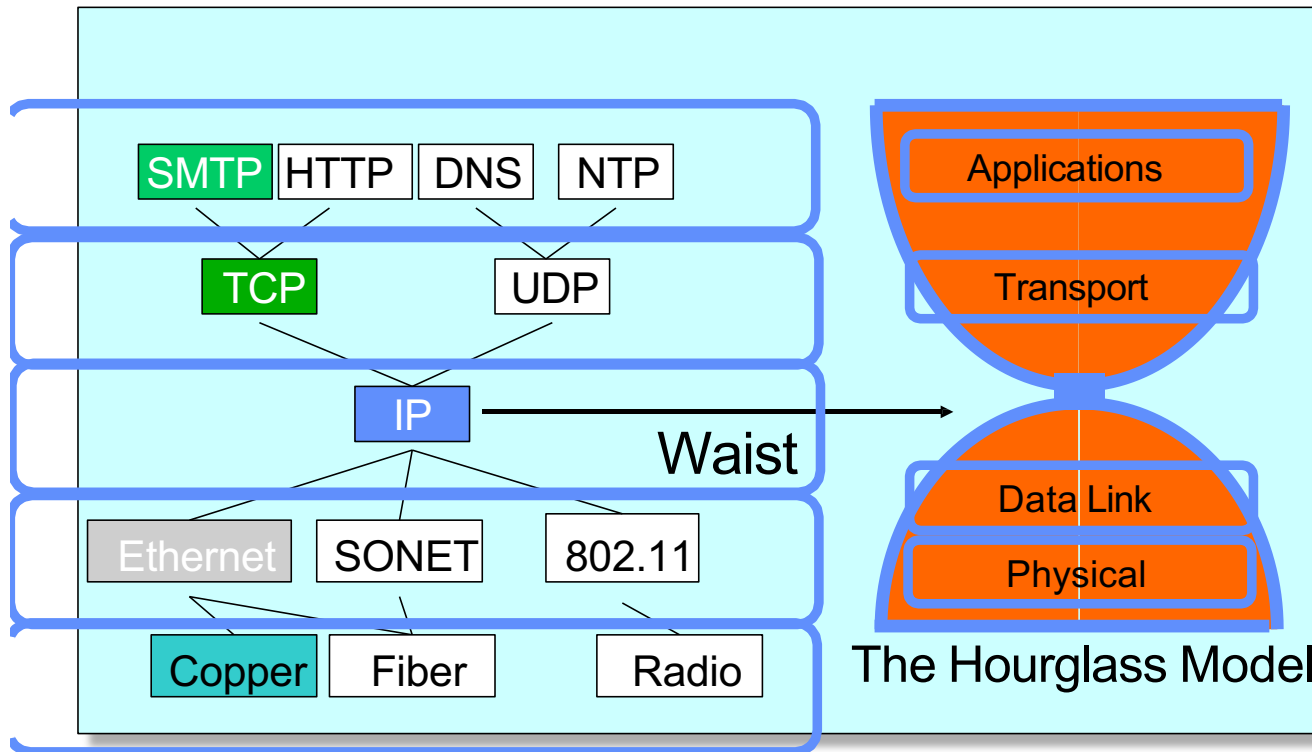


The Internet: layers, layers, layers



- Introduce intermediate layers that provide **set of abstractions** for various network functionality & technologies
 - A new app/media implemented only once
- Goal: reliable communication channels on which to build distributed applications

The Internet: hourglass model



- “Narrow waist” facilitates **interoperability**
 - Layers “abstract” away hardware so that upper layers are agnostic to lower layers
- => Sound familiar?

The Internet: implications of hourglass

- Single Internet-layer module (**IP**):
 - Allows arbitrary networks to interoperate
 - Any network technology that supports IP can exchange packets
 - Allows applications to function on all networks
 - Applications that can run on IP can **use any network**
 - Supports simultaneous innovations above and below IP
 - But changing IP itself, i.e., **IPv6**, very involved

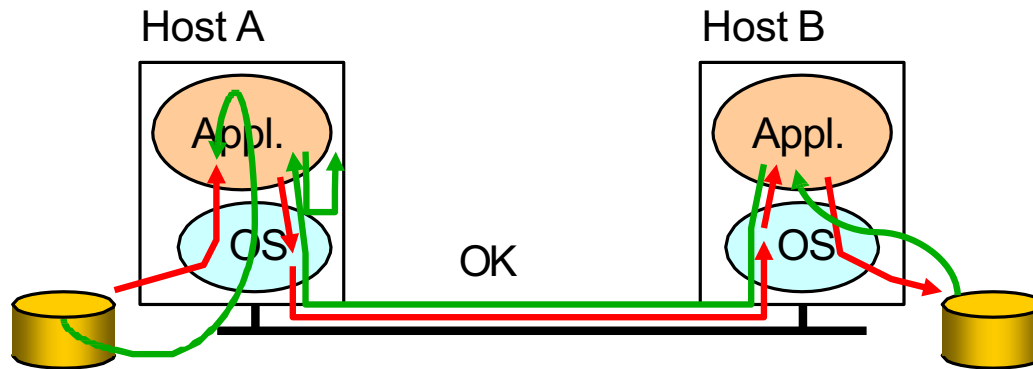
The Internet: drawbacks of layering

- Layer N may duplicate layer N-1 functionality
 - E.g., error recovery to retransmit lost data
- Layers may need same information
 - E.g., timestamps, maximum transmission unit size
- Layering can hurt performance
 - E.g., hiding details about what is really going on
- Some layers are not always cleanly separated
 - Inter-layer dependencies for performance reasons
 - Some dependencies in standards (header checksums)

End-to-end argument

- Hugely influential paper:
 - “End-to-End Arguments in System Design” by Saltzer, Reed, and Clark (‘84)
- “Sacred Text” of the Internet
 - Endless disputes about what it means
 - Everyone cites it as supporting their position
- Simple Message: Some types of network functionality can only be correctly implemented **end-to-end**
 - Reliability, security, etc.
 - Hosts cannot rely on the network help to meet requirement, so must implement it themselves

Example: reliable file transfer



- Solution 1: make each step reliable, and then concatenate them
- Solution 2: end-to-end check and try again if necessary

Discussion

- Solution 1 is incomplete
 - What happens if memory is corrupted? Receiver has to do the check anyway!
- Solution 2 is complete
 - Full functionality can be entirely implemented at application layer with no need for reliability from lower layers
- *Is there any need to implement reliability at lower layers?*
 - Well, it could be more efficient

End-to-end principle

- Implementing complex functionality in the network:
 - Doesn't always reduce host implementation complexity
 - Does increase network complexity
 - Probably imposes delay and overhead on all applications, even if they don't need functionality
- However, implementing in network can enhance performance in some cases
 - – e.g., very lossy link

Conservative interpretation of E2E

- Don't implement a function at the lower levels of the system unless it can be completely implemented at this level

Moderate interpretation

- Think twice before implementing functionality in the network
- If hosts can implement functionality correctly, implement it in a lower layer only as a performance enhancement
- But do so only if it does not impose burden on applications that do not require that functionality

Thanks